

Ch7. Gradient Descent

Section 7.3. Convergence

第 7 章-梯度下降 – 第 7.3 节-收敛性

《深度学习基础》课程小组

2026 年 8 月 1 日

1 P1

When applying gradient descent in practice, we need to choose a value for the learning rate parameter η . Consider the simple error surface depicted in **Figure 7.3** for a hypothetical two-dimensional weight space in which the curvature of E varies significantly with direction, creating a ‘valley’. At most points on the error surface, the local gradient vector for batch gradient descent, which is perpendicular to the local contour, does not point directly towards the minimum. Intuitively we might expect that increasing the value of η should lead to bigger steps through weight space and hence faster convergence. However, the successive steps oscillate back and forth across the valley, and if we increase η too much, those oscillations will become divergent. Because η must be kept sufficiently small to avoid divergent oscillations across the valley, progress along the valley is very slow. Gradient descent then takes many small steps to reach the minimum and is a very inefficient procedure.

Figure 7.3 Caption: Schematic illustration of fixed-step gradient descent for an error function that has substantially different curvatures along different directions. The error surface E has the form of a long valley, as depicted by the ellipses. Note that, for most points in weight space, the local negative gradient vector $-\nabla E$ does not point towards the minimum of the error function. Successive steps of gradient descent can therefore oscillate across the valley, leading to very slow progress along the valley towards the minimum. The vectors u_1 and u_2 are the eigenvectors of the Hessian matrix.

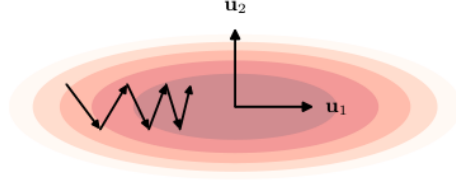


图 1: 步长固定的梯度下降

2 P2

We can gain deeper insight into the nature of this problem by considering the quadratic approximation to the error function in the neighbourhood of the minimum. From (7.7), (7.8), and (7.10), the gradient of the error function in this approximation can be written as

$$\nabla E = \sum_i \alpha_i \lambda_i \mathbf{u}_i. \quad (7.24)$$

Again using (7.10) we can express the change in the weight vector in terms of corresponding changes in the coefficients $\{\alpha_i\}$:

$$\Delta \mathbf{w} = \sum_i \Delta \alpha_i \mathbf{u}_i. \quad (7.25)$$

Combining (7.24) with (7.25) and the gradient descent formula (7.16) and using the orthonormality relation (7.9) for the eigenvectors of the Hessian, we obtain the following expression for the change in α_i at each step of the gradient descent algorithm:

$$\Delta \alpha_i = -\eta \lambda_i \alpha_i \quad (7.26)$$

from which it follows that

$$\alpha_i^{\text{new}} = (1 - \eta \lambda_i) \alpha_i^{\text{old}} \quad (7.27)$$

where ‘old’ and ‘new’ denote values before and after a weight update. Using the orthonormality relation (7.9) for the eigenvectors together with (7.10), we have

$$\mathbf{u}_i^T (\mathbf{w} - \mathbf{w}^*) = \alpha_i \quad (7.28)$$

and so α_i can be interpreted as the distance to the minimum along the direction $\mathbf{u}_i$. From (7.27) we see that these distances evolve independently such that, at each step, the distance along the direction of $\mathbf{u}_i$ is multiplied by a factor $(1 - \eta \lambda_i)$. After a total of T steps we have

$$\alpha_i^{(T)} = (1 - \eta \lambda_i)^T \alpha_i^{(0)}. \quad (7.29)$$

It follows that, provided $|1 - \eta\lambda_i| < 1$, the limit $T \rightarrow \infty$ leads to $\alpha_i = 0$, which from (7.28) shows that $\mathbf{w} = \mathbf{w}^*$ and so the weight vector has reached the minimum of the error.

3 P3

Note that (7.29) demonstrates that gradient descent leads to linear convergence in the neighbourhood of a minimum. Also, convergence to the stationary point requires that all the λ_i be positive, which in turn implies that the stationary point is indeed a minimum. By making η larger we can make the factor $(1 - \eta\lambda_i)$ smaller and hence improve the speed of convergence. There is a limit to how large η can be made, however. We can permit $(1 - \eta\lambda_i)$ to go negative (which gives oscillating values of α_i), but we must ensure that $|1 - \eta\lambda_i| < 1$ otherwise the α_i values will diverge. This limits the value of η to $\eta < 2/\lambda_{\max}$ where $\lambda_{\max}$ is the largest of the eigenvalues. The rate of convergence, however, is dominated by the smallest eigenvalue, so with η set to its largest permitted value, the convergence along the direction corresponding to the smallest eigenvalue (the long axis of the ellipse in **Figure 7.3**) will be governed by

$$\left(1 - \frac{2\lambda_{\min}}{\lambda_{\max}}\right) \quad (7.30)$$

where $\lambda_{\min}$ is the smallest eigenvalue. If the ratio $\lambda_{\min}/\lambda_{\max}$ (whose reciprocal is known as the *condition number* of the Hessian) is very small, corresponding to highly elongated elliptical error contours as in **Figure 7.3**, then progress towards the minimum will be extremely slow.

4 P4 7.3.1 Momentum

One simple technique for dealing with the problem of widely differing eigenvalues is to add a *momentum* term to the gradient descent formula. This effectively adds inertia to the motion through weight space and smooths out the oscillations depicted in **Figure 7.3**. The modified gradient descent formula is given by

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta \nabla E(\mathbf{w}^{(\tau-1)}) + \mu \Delta \mathbf{w}^{(\tau-2)} \quad (7.31)$$

where μ is called the momentum parameter. The weight vector is then updated using (7.15).

To understand the effect of the momentum term, consider first the motion through a region of weight space for which the error surface has relatively low curvature, as indicated in **Figure 7.4**. If we make

the approximation that the gradient is unchanging, then we can apply (7.31) iteratively to a long series of weight updates, and then sum the resulting arithmetic series to give

$$\Delta \mathbf{w} = -\eta \nabla E \{1 + \mu + \mu^2 + \dots\} \quad (1)$$

$$= -\frac{\eta}{1 - \mu} \nabla E \quad (7.32)$$

$$(7.33)$$

and we see that the result of the momentum term is to increase the effective learning rate from η to $\eta/(1 - \mu)$.

Figure 7.4 Caption: With a fixed learning rate parameter, gradient descent down a surface with low curvature leads to successively smaller steps corresponding to linear convergence. In such a situation, the effect of a momentum term is like an increase in the effective learning rate parameter.

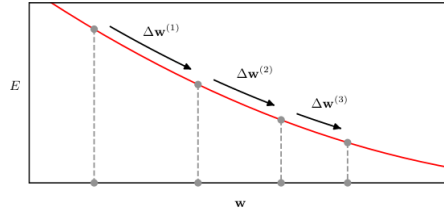


图 2: 误差曲面的曲率较低的情形

5 P5

By contrast, in a region of high curvature in which gradient descent is oscillatory, as indicated in **Figure 7.5**, successive contributions from the momentum term will tend to cancel and the effective learning rate will be close to η . Thus, the momentum term can lead to faster convergence towards the minimum without causing divergent oscillations. A schematic illustration of the effect of a momentum term is shown in **Figure 7.6**.

Figure 7.5 Caption: For a situation in which successive steps of gradient descent are oscillatory, a momentum term has little influence on the effective value of the learning rate parameter.

Figure 7.6 Caption: Illustration of the effect of adding a momentum term to the gradient descent algorithm, showing the more rapid progress along the valley of the error function, compared with the unmodified gradient descent shown in Figure 7.3.

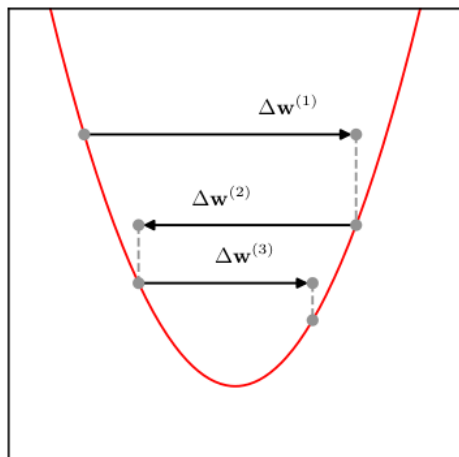


图 3: 误差曲面有较高曲率的情形

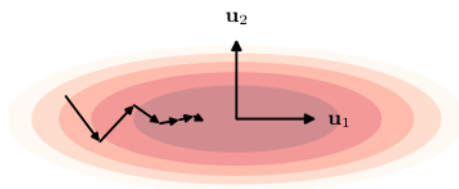


图 4: 添加动量项的梯度下降

6 P6

Although the inclusion of momentum can lead to an improvement in the performance of gradient descent, it also introduces a second parameter μ whose value needs to be chosen, in addition to that of the learning rate parameter η . From (7.33) we see that μ should be in the range $0 \leq \mu \leq 1$. A typical value used in practice is $\mu = 0.9$. Stochastic gradient descent with momentum is summarized in Algorithm 7.3.

Algorithm 1 7.3. Stochastic gradient descent with momentum

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$

Batch size B

Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$

Learning rate parameter η

Momentum parameter μ

Initial weight vector $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

```
 $n \leftarrow 1$ 
 $\Delta \mathbf{w} \leftarrow 0$ 
repeat
   $\Delta \mathbf{w} \leftarrow -\eta \nabla E_{n:n+B-1}(\mathbf{w}) + \mu \Delta \mathbf{w}$  // 带动量的更新
   $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$  // 更新权重向量
   $n \leftarrow n + B$ 
  if  $n + B > N$  then
    shuffle data
     $n \leftarrow 1$ 
  end if
until convergence
return  $\mathbf{w}$ 
```

7 P7

The convergence can be further accelerated using a modified version of momentum called *Nesterov momentum* (Nesterov, 2004; Sutskever et al., 2013). In conventional stochastic gradient descent with momentum, we first compute the gradient at the current location then take a step that is amplified by adding momentum from the previous step. With the Nesterov method, we change the order of these and first compute a step based on the previous momentum, then calculate the gradient at this new location to find the update, so that

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta \nabla E(\mathbf{w}^{(\tau-1)} + \mu \Delta \mathbf{w}^{(\tau-2)}) + \mu \Delta \mathbf{w}^{(\tau-2)}. \quad (7.34)$$

For batch gradient descent, Nesterov momentum can improve the rate of convergence, although for stochastic gradient descent it can be less effective.

8 P8 7.3.2 Learning rate schedule

In the stochastic gradient descent learning algorithm (7.18), we need to specify a value for the learning rate parameter η . If η is very small then learning will proceed slowly. However, if η is increased too

much it can lead to instability. Although some oscillation can be tolerated, it should not be divergent. In practice, the best results are obtained by using a larger value for η at the start of training and then reducing the learning rate over time, so that the value of η becomes a function of the step index τ :

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta^{(\tau-1)} \nabla E_n(\mathbf{w}^{(\tau-1)}). \quad (7.35)$$

9 P9

Examples of learning rate schedules include linear, power law, and exponential decay:

$$\eta^{(\tau)} = (1 - \tau/K) \eta^{(0)} + (\tau/K) \eta^{(K)} \quad (7.36)$$

$$\eta^{(\tau)} = \eta^{(0)} (1 + \tau/s)^c \quad (7.37)$$

$$\eta^{(\tau)} = \eta^{(0)} c^{\tau/s} \quad (7.38)$$

where in (7.36) the value of η reduces linearly over K steps, after which its value is held constant at $\eta^{(K)}$. Good values for the hyperparameters $\eta^{(0)}$, $\eta^{(K)}$, K , S , and c must be found empirically. It can be very helpful in practice to monitor the *learning curve* showing how the error function evolves during the gradient descent iteration to ensure that it is decreasing at a suitable rate.

10 P10 7.3.3 RMSProp and Adam

We saw that the optimal learning rate depends on the local curvature of the error surface, and moreover that this curvature can vary according to the direction in parameter space. This motivates several algorithms that use different learning rates for each parameter in the network. The values of these learning rates are adjusted automatically during training. Here we review some of the most widely used examples. Note, however, that this intuition really applies only if the principal curvature directions are aligned with the axes in weight space, corresponding to a locally diagonal Hessian matrix, which is unlikely to be the case in practice. Nevertheless, these types of algorithms can be effective and are widely used.

11 P11

The key idea behind *AdaGrad*, short for ‘adaptive gradient’, is to reduce each learning rate parameter over time by using the accumulated sum of squares of all the derivatives calculated for that parameter (Duchi, Hazan, and Singer, 2011). Thus, parameters associated with high curvature are reduced most rapidly. Specifically,

$$r_i^{(\tau)} = r_i^{(\tau-1)} + \left(\frac{\partial E(\mathbf{w})}{\partial w_i} \right)^2 \quad (7.39)$$

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)}} + \delta} \left(\frac{\partial E(\mathbf{w})}{\partial w_i} \right) \quad (7.40)$$

where η is the learning rate parameter, and δ is a small constant, say 10^{-8} , that ensures numerical stability in the event that r_i is close to zero. The algorithm is initialized with $r_i^{(0)} = 0$. Here $E(\mathbf{w})$ is the error function for a particular mini-batch, and the update (7.40) is standard stochastic gradient descent but with a modified learning rate that is specific to each parameter.

12 P12

One problem with AdaGrad is that it accumulates the squared gradients from the very start of training, and so the associated weight updates can become very small, which can slow down training too much in the later phases. The idea behind the *RMSPprop* algorithm, which is short for ‘root mean square propagation’, is to replace the sum of squared gradients of AdaGrad with an exponentially weighted average (Hinton, 2012), giving

$$r_i^{(\tau)} = \beta r_i^{(\tau-1)} + (1 - \beta) \left(\frac{\partial E(\mathbf{w})}{\partial w_i} \right)^2 \quad (7.41)$$

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)}} + \delta} \left(\frac{\partial E(\mathbf{w})}{\partial w_i} \right) \quad (7.42)$$

where $0 < \beta < 1$ and a typical value is $\beta = 0.9$.

13 P13

If we combine RMSProp with momentum, we obtain the *Adam* optimization method (Kingma and Ba, 2014) where the name is derived from ‘adaptive moments’. Adam stores the momentum for each parameter separately using update equations that consist of exponentially weighted moving averages for both the gradients and the squared gradients in the form

$$s_i^{(\tau)} = \beta_1 s_i^{(\tau-1)} + (1 - \beta_1) \left(\frac{\partial E(\mathbf{w})}{\partial w_i} \right) \quad (7.43)$$

$$r_i^{(\tau)} = \beta_2 r_i^{(\tau-1)} + (1 - \beta_2) \left(\frac{\partial E(\mathbf{w})}{\partial w_i} \right)^2 \quad (7.44)$$

$$\hat{s}_i^{(\tau)} = \frac{s_i^{(\tau)}}{1 - \beta_1^\tau} \quad (7.45)$$

$$\hat{r}_i^{(\tau)} = \frac{r_i^{(\tau)}}{1 - \beta_2^\tau} \quad (7.46)$$

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \eta \frac{\hat{s}_i^\tau}{\sqrt{\hat{r}_i^\tau + \delta}}. \quad (7.47)$$

Here the factors $1/(1 - \beta_1^\tau)$ and $1/(1 - \beta_2^\tau)$ correct for a bias introduced by initializing $s_i^{(0)}$ and $r_i^{(0)}$ to zero. Note that the bias goes to zero as τ becomes large, since $\beta_i < 1$, and so in practice this bias correction is sometimes omitted. Typical values for the weighting parameters are $\beta_1 = 0.9$ and $\beta_2 = 0.99$. Adam is the most widely adopted learning algorithm in deep learning and is summarized in Algorithm 7.4.

Algorithm 2 7.4. Adam optimization

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$

Batch size B

Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$

Learning rate parameter η

Decay parameters β_1 and β_2

Stabilization parameter δ

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$

$s \leftarrow 0$

$r \leftarrow 0$

repeat

 Choose a mini-batch at random from D

$g = -\nabla E_{n:n+B-1}(w)$

// 计算梯度向量

$s \leftarrow \beta_1 s + (1 - \beta_1)g$

$r \leftarrow \beta_2 r + (1 - \beta_2)g \odot g$

// 按分量对应相乘

$\hat{s} \leftarrow s / (1 - \beta_1^t)$

$\hat{r} \leftarrow r / (1 - \beta_2^t)$

$\Delta \mathbf{w} \leftarrow -\eta \frac{\hat{s}}{\sqrt{\hat{r} + \delta}}$

$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$

// 更新权重向量

$n \leftarrow n + B$

if $n + B > N$ **then**

 shuffle data

$n \leftarrow 1$

end if

until convergence

return $\mathbf{w}$
